Provider–Observer–Operator: A Role-Based Abstraction Discipline for Interface-Led Architecture

Takuya Sogawa

Contents

Provider–Observer–Operator

A Role-Based Abstraction Discipline for Interface-Led Architecture

Version: v1.0.0
Type: Technical Note / ILA Supplement
Author: Takuya Sogawa
ORCID: https://orcid.org/0009-0009-7499-2595
DOI: 10.5281/zenodo.20322690

Abstract

This technical note defines **Provider–Observer–Operator** as a role-based abstraction discipline for applying Interface-Led Architecture (ILA) to concrete software design and refactoring.

ILA defines software architecture from interfaces and contracts rather than from implementation classes, framework layers, inheritance hierarchies, or incidental technology choices. This paper supplements the existing ILA methodology by defining how interfaces are extracted: every interface-bearing

component is first classified as one of three primitive roles—**Provider**, **Observer**, or **Operator**. Higher-order structures are treated as governed compositions of these roles and are called **Units**. Units are then connected by **Pipelines** operating under **Core Contracts**.

The central claim of this paper is that interface design should abstract not reusable implementation, but supplied capabilities, observed evidence, executable operations, and the contract boundaries that connect them. A Provider supplies capabilities. An Observer observes evidence. An Operator transforms or executes state. A Unit composes roles. A Pipeline orders Units. Core Contracts constrain their interaction.

This paper also clarifies that exceptions are not roles. An exception is a failure state or transition result that occurs when a Pipeline can no longer proceed under its Core Contract. Optional metadata remains optional unless it is required for contractual correctness.

Semantic IR, Composite Distance, Governed Circuits, and ReplayLog are not part of the general ILA theory. They are AIKernel-specific specializations that appear when ILA is applied to semantic runtime governance. By separating the general role model from the AIKernel specialization, this paper keeps ILA broadly applicable while preserving its connection to the AIKernel architecture.

Keywords

Interface-Led Architecture; ILA; Provider; Observer; Operator; Unit; Pipeline; Core Contract; Role-Based Abstraction; Software Architecture; AIKernel; Contract-First Design; Fail-Closed; Capability-Based Design

---

1. Introduction

Interface-Led Architecture (ILA) is a software development methodology that places interfaces and contracts before implementation. Its purpose is not merely to increase reuse, but to make architectural boundaries explicit, inspectable, and enforceable.

However, when ILA is applied to an existing codebase, a practical question arises: **how should interfaces be discovered?** If the answer is based only on implementation similarity, framework layering, or anticipated reuse, the resulting abstraction often becomes unstable. Large interfaces appear. Unsupported methods become normal. Policies leak into providers. Observers mutate state. Exceptions become control flow. Optional metadata becomes accidentally mandatory.

This paper introduces a compact abstraction discipline for avoiding these failures. It argues that interface extraction should begin by classifying components by role:

- **Provider** —supplies something;
- **Observer** —observes something;
- **Operator** —acts upon something.

Higher-level architectural structures are not new primitive roles. They are governed role compositions. This paper calls such compositions **Units**. Units are connected by **Pipelines**, and Pipelines are valid only when their transitions are defined by **Core Contracts**.

The resulting model is intentionally small. It is designed to be usable during refactoring: for each component, the architect asks whether it supplies, observes, or operates. If it does several of these, it should be decomposed or treated as a Unit. If a component cannot be described in these terms, the design boundary is likely unclear.

2. Relationship to Interface-Led Architecture

This paper is a supplement to the existing Interface-Led Architecture methodology. The ILA paper defines the larger architectural stance: software should be structured by interface contracts, responsibility

boundaries, and deterministic execution contexts rather than by implementation-first generation.

The present paper answers a narrower question: **what is the abstraction discipline used to extract those interfaces?**

In that sense, the relationship is as follows:

```
Interface-Led Architecture:
  Build software from interfaces, contracts, and responsibility boundaries.

Provider–Observer–Operator Discipline:
  Extract those interfaces by classifying components into primitive roles,
  composing them into Units, and connecting Units through Pipelines.
```

Thus, this paper does not replace ILA. It makes ILA operational for refactoring and implementation design.

3. Problem Statement

Common abstraction strategies often begin from the wrong source of similarity. They ask whether two classes share implementation, whether they belong to the same framework layer, or whether future reuse is expected. These questions are useful, but they are insufficient for interface design.

Two components may be implemented similarly while having different contractual responsibilities. Conversely, two components may be implemented differently while exposing the same contractual role.

For example, a file component may read, write, delete, enumerate, observe, snapshot, and audit. If these operations are grouped into a single large interface, read-only providers are forced to expose unsupported operations. The system then depends on runtime rejection rather than on contract-level separation.

Typical symptoms include:

- oversized interfaces;
- normal use of `NotSupportedException`;
- providers that contain policy decisions;
- observers that mutate state;
- operators that silently depend on optional metadata;
- exception handling used as ordinary control flow;
- pipeline steps connected by implementation knowledge rather than Core Contracts.

ILA requires a discipline that distinguishes role, contract, and composition before implementation details are considered.

4. Primitive Role Model

The minimal abstraction unit in ILA is not a class, package, module, or framework layer. It is a **Role**.

A Role describes the contractual responsibility that a component plays in a software system. From the perspective of interface extraction, ILA classifies roles into three primitives:

```
Role ::= Provider | Observer | Operator
```

This is not a claim that all entities in the world are ontologically reducible to these three categories. The claim is more precise: for interface extraction and responsibility separation, these three roles are sufficient primitives.

4.1 Provider

A **Provider** supplies data, state, capability, resource access, services, or executable targets through a contract.

A Provider declares what it can provide. It should not decide whether a higher-level action is safe, authorized, aligned, or semantically acceptable. Those decisions belong to policies, gates, operators, or higher-level Units.

Examples include:

```
MemoryProvider
ModelProvider
StateProvider
VfsProvider
EmbeddingProvider
ClockProvider
ReplayLogProvider
```

The Provider rule is:

```
Provider provides capabilities.
Provider does not govern.
```

4.2 Observer

An **Observer** observes state, change, metrics, risk, drift, events, logs, or evidence.

An Observer produces evidence. It should not directly mutate the system state it observes. Observers may feed Policy Decision Points, Operators, ReplayLogs, diagnostics, or audit systems, but observation and mutation must remain separated.

Examples include:

```
StateObserver
RuntimeObserver
TrajectoryObserver
RiskObserver
DriftObserver
ReplayObserver
MetricObserver
```

The Observer rule is:

```
Observer observes evidence.
Observer does not mutate.
```

4.3 Operator

An **Operator** acts on input, state, resources, or semantic structures. It may read, write, run, infer, transform, validate, compile, synthesize, project, route, or decide.

In ILA, “use,”“execute,”“transform,”and “decide”are all forms of operation. Because Operators may cause state transitions or side effects, they require stronger contracts than Providers or Observers.

Examples include:

```
ReadOperator
WriteOperator
InferenceOperator
TransformOperator
ValidationOperator
SynthesisOperator
AdmissibilityGate
PolicyDecisionOperator
```

The Operator rule is:

```
Operator transforms state.
Operator must be governed.
```

5. Unit as Governed Composition

Higher-order structures are not fourth primitive roles. Runtime, Layer, Pipeline, Module, Subsystem, or Service are often better understood as compositions of Provider, Observer, and Operator roles.

This paper calls such a governed composition a **Unit**.

```
Unit = governed composition of Providers, Observers, and Operators
```

A Unit is the smallest semantic composition that can participate in a Pipeline as a meaningful architectural block.

5.1 Examples

```
MemoryUnit
├── Provider: MemoryProvider
└── Operator: Read / Write / Snapshot
```

```
RuntimeUnit
├── Provider: RuntimeStateProvider
├── Observer: RuntimeObserver
└── Operator: ExecutionOperator
```

```
PipelineUnit
├── Observer: PipelineStateObserver
└── Operator: PipelineStepOperator
```

```
ModelUnit
├── Provider: ModelProvider
├── Observer: TokenUsageObserver / LatencyObserver
└── Operator: InferenceOperator
```

In AIKernel, a VFS Unit may be represented as:

```
VFS Unit
├── Provider: IVfsProvider
├── Observer: IVfsStateObserver / IVfsAuditObserver
└── Operator: IReadableVfsSession / IWritableVfsSession
```

A Trajectory Governance Unit may be represented as:

```
Trajectory Governance Unit
├── Provider: CandidateProvider / ContextProvider
├── Observer: TrajectoryObserver / RiskObserver
└── Operator: TrajectoryGovernor / PolicyDecisionOperator
```

5.2 Unit Properties

A Unit has the following properties:

- it is defined by a composition of primitive roles;
- it prioritizes interfaces and contracts over implementation;
- it operates through Core Contracts;
- it is connected to other Units through Pipelines;
- it may expose optional metadata;
- optional metadata must not be required for contractual correctness.

6. Pipeline and Core Contract

A Pipeline is an ordered composition of Units.

```
Pipeline = Unit_1 -> Unit_2 -> ... -> Unit_n
```

However, a Pipeline is not merely a sequence of method calls. In ILA, a Pipeline is valid only when each transition is defined by a **Core Contract**.

A Core Contract is the minimum contractual boundary required for a Unit transition. It may include:

- input contract;
- output contract;
- capability declaration;
- failure mode;
- determinism requirement;
- security boundary;
- replay requirement.

In AIKernel, this produces a structure such as:

```
ROM Unit
  -> VFS Unit
  -> Pre-Inference Governance Unit
  -> Trajectory Governance Unit
  -> Execution Unit
  -> Replay Unit
```

Each Unit does not need to know the implementation of the next Unit. It only needs to satisfy the Core Contract required by the Pipeline.

7. Exception as State, Not Role

An exception is not a role.

Exceptions do not supply, observe, or operate as primitive architectural responsibilities. Instead, an exception is a failure state or transition result that occurs when a Pipeline can no longer proceed under its Core Contract.

```
Exception is not a role.
Exception is a failure state of a Pipeline or Contract transition.
```

This distinction is important. If exceptions are treated as ordinary control flow, the architecture begins to depend on runtime escape paths rather than contract-level validity. In an ILA system, failure should be represented as a contractual state whenever possible.

For AIKernel, this aligns with Fail-Closed design. When a Pipeline cannot safely continue, the system should record the failure state and transition to denial, clarification, abort, or safe fallback rather than silently continuing.

8. Optional Metadata Rule

All non-contractual metadata is optional.

A Core Contract should contain only what is required for correctness. Additional information may be useful for diagnostics, documentation, routing hints, user interfaces, observability, or provider-specific optimization, but it must not be required for contractual validity unless it is promoted into the contract.

Required examples:

```
Role
Input
Output
```

```
Capability
Failure mode
Contract boundary
```

Optional examples:

```
description
tags
display name
diagnostics
statistics
provider-specific hints
UI hints
documentation metadata
```

If metadata is required for a Pipeline to be correct, it is no longer metadata. It is part of the Core Contract.

9. Relationship to Reactive Architecture

Reactive architectures emphasize observation, propagation, asynchronous event handling, and response to change. This perspective is valuable because it foregrounds observability and dynamic propagation.

ILA does not reject Reactive architecture. It generalizes the observer-centered view into a role-complete abstraction model by separating Provider, Observer, and Operator as distinct interface roles.

```
Reactive architecture:
  observation and propagation are emphasized.

ILA role model:
  Provider, Observer, and Operator are separated as primitive roles.
```

In this sense, ILA absorbs the strength of the Reactive perspective while avoiding the reduction of the entire architecture to observation alone.

10. General ILA Core and AIKernel Specialization

The Provider–Observer–Operator model belongs to the general ILA theory.

```
ILA General Core
├── Provider
├── Observer
├── Operator
├── Unit
├── Pipeline
└── Core Contract
```

Semantic IR, Composite Distance, Governed Circuits, ReplayLog, Semantic State, and Runtime Policy are not part of the general ILA theory. They appear when ILA is specialized for AIKernel and semantic runtime governance.

```
AIKernel Specialization
├── Semantic IR
├── Composite Distance
├── Governed Circuit
├── ReplayLog
├── Semantic State
└── Runtime Policy
```

This separation protects the generality of ILA. If Semantic IR or Composite Distance were treated as part of ILA itself, ILA would become an AIKernel-specific theory. Instead, ILA remains a general interface-design methodology, while AIKernel becomes one of its governed semantic-runtime applications.

11. Axioms of ILA Abstraction

The abstraction discipline can be summarized as the following axioms.

Axiom 1: Primitive Role Classification

Every interface-bearing component is classified as Provider, Observer, Operator, or a Unit composed of them.

Axiom 2: Unit Is Not a Fourth Primitive Role

A Unit is not a fourth primitive role. It is a governed composition of primitive roles.

Axiom 3: Pipeline Is Ordered Unit Composition

A Pipeline is an ordered composition of Units operating under Core Contracts.

Axiom 4: Exception Is Not a Role

An exception is not a role. It is a failure state of a Pipeline or Contract transition.

Axiom 5: Optional Metadata Must Remain Optional

Additional metadata is optional unless it is required for contractual correctness. If it is required for correctness, it must be promoted into the Core Contract.

12. Practical Refactoring Guidelines

When refactoring an existing implementation under ILA, the following questions should be asked in order:

1. What does this component provide?
2. What does this component observe?
3. What does this component operate on, transform, execute, or decide?
4. Is this a primitive role or a Unit composed of multiple roles?
5. Are Unit transitions expressed through Core Contracts?
6. Are policy decisions mixed into Providers?
7. Are Observers mutating state?
8. Are Operators governed by explicit contracts?
9. Are exceptions used as normal control flow?
10. Has optional metadata become accidentally mandatory?
11. Does the design require `NotSupportedException` as a normal path?
12. Can the Pipeline fail closed?

A useful warning sign is the presence of `NotSupportedException` in an interface implementation. It often indicates that a contract is too large or that role separation is incomplete.

13. Limitations and Non-Claims

This paper does not claim that every entity in software, computation, or the world is ontologically reducible to Provider, Observer, and Operator.

The claim is architectural and methodological: from the perspective of interface extraction and responsibility separation, these three roles provide a compact and sufficient basis for designing contracts.

This paper also does not define Semantic IR, Composite Distance, Semantic Ellipsoid, Governed Circuits, or ReplayLog as general ILA concepts. Those belong to the AIKernel specialization.

Finally, this paper does not reject Reactive architecture. It repositions the observer-centered insight of reactive systems within a broader role model that also includes supply and operation.

14. Conclusion

This paper defined Provider–Observer–Operator as a role-based abstraction discipline for Interface-Led Architecture.

ILA interface design begins from three primitive roles: Provider, Observer, and Operator. Their governed compositions are Units. Units are connected by Pipelines. Pipelines operate under Core Contracts. Additional metadata remains optional unless required for contractual correctness.

Exceptions are not roles. They are failure states that occur when a Pipeline cannot continue under its Core Contract.

Semantic IR and Composite Distance are not part of the general ILA theory. They are AIKernel-specific specializations that appear when ILA is applied to semantic runtime governance.

With this discipline, ILA becomes not only a philosophy of interface-first design, but a practical method for refactoring software systems into roles, Units, Pipelines, and enforceable contracts.

References

1. Sogawa, Takuya. “Interface-Led Architecture (ILA): A Software Development Methodology for the AI Era, Validated by the AIKernel Execution Model.”Zenodo, 2026. DOI: 10.5281/zenodo.20290614.
2. Gamma, Erich, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
3. Bonér, Jonas, Dave Farley, Roland Kuhn, and Martin Thompson. “The Reactive Manifesto.”2014. Available at: https://www.reactivemanifesto.org/.